Diluted Steganography: How Signal Dilution Degrades LLM Steganographic Capabilities

Ari Holtzman and NeuriCo

Abstract

Large language models can produce near-perfect acrostic steganography, where the first letter of each sentence spells a hidden message. But does this reflect a general steganographic capability, or merely a memorized pattern? We investigate by systematically increasing signal dilution: instead of encoding one character per sentence, we test schemes that embed characters at every 3rd, 5th, 10th, and 20th word position, as well as a novel trigger-word scheme where the word following each occurrence of "the" carries the signal. We evaluate three frontier models (GPT-4.1, Claude Sonnet 4.5, Gemini 2.5 Pro) on encoding, informed extraction, and blind detection across these six schemes (270 total trials). All three models achieve 92–100% per-character accuracy on acrostics but collapse to 0–43% on every diluted scheme—a sharp cliff rather than a gradual decline. Informed extraction shows a similar pattern: GPT-4.1 retains partial accuracy (35–55%) on diluted schemes, while Claude and Gemini degrade more sharply. Blind detection fails entirely: models either flag all texts or no texts as steganographic, with zero discriminability at every dilution level. These results demonstrate that current LLM steganographic ability is pattern-specific, not compositional, and that even minimal dilution beyond acrostics defeats both production and detection. This has direct implications for AI safety monitoring systems that assume LLM-based detectors can identify covert communication.

1 Introduction

Can large language models hide messages in plain text? Recent work suggests the answer is a qualified yes: frontier models produce convincing acrostic encodings where the first letter of each sentence spells a secret message, sometimes achieving over 90% per-character accuracy [Karpov, 2026, Zolkowski et al., 2025]. This capability raises immediate concerns for AI safety. If models can encode information steganographically, they could potentially embed hidden reasoning in chain-of-thought outputs [Roger and Greenblatt, 2023], coordinate covertly in multi-agent systems [Motwani et al., 2024], or smuggle information past monitoring systems [Mathew et al., 2024].

But acrostics are a special case. They align signal with sentence boundaries—a natural linguistic unit that appears frequently in training data through poems, puzzles, and wordplay exercises. The practical question for AI safety is not whether models can produce acrostics, but whether they possess a *general* steganographic capability that extends to more diluted encoding schemes. If a model can hide a message in the first letters of every 10th or 20th word, evading detection becomes far easier because the signal is spread thinly across a larger text.

Prior work has begun to address this question but leaves significant gaps. Karpov [2026] systematically varied sequence length within acrostic encoding, showing that accuracy degrades as messages grow longer. Zolkowski et al. [2025] compared acrostic (`sentence_letter`) and every-third-word (`third_word_letter`) schemes, finding that models perform worse on the latter. However, no study has systematically varied the dilution level as a continuous parameter across a wide range, measured

encoding and extraction and detection in a unified framework, or tested trigger-word-based schemes where signal placement depends on lexical context rather than fixed positions.

We address this gap with a systematic study of signal dilution in LLM steganography. We test three frontier models—GPT-4.1, CLAUDE SONNET 4.5, and GEMINI 2.5 PRO—across six encoding schemes spanning a dilution gradient from concentrated acrostics ($\sim$1 character per sentence) to highly diluted every-20th-word patterns ($\sim$1 character per 20 words), including a novel trigger-word scheme. We evaluate three capabilities: (1) encoding—can the model produce text with the hidden message? (2) informed extraction—can the model recover the message when told the scheme? (3) blind detection—can the model detect that a message is present without knowing the scheme?

Our main finding is a *cliff effect*: all three models drop from 92–100% accuracy on acrostics to below 43% on every other scheme, with no gradual degradation between dilution levels. This is not a smooth capability curve but an abrupt transition from a well-learned pattern to near-random performance. For blind detection, the picture is even starker: no model can discriminate between steganographic and control texts at any dilution level.

Our contributions are:

- We conduct the first systematic study of signal dilution in LLM steganography, testing six encoding schemes across three frontier models in a unified evaluation framework covering encoding, extraction, and detection (270 total trials).
- We identify a cliff effect in steganographic capability: performance collapses abruptly beyond acrostics rather than degrading gradually, suggesting acrostic ability is a memorized pattern rather than a general compositional skill.
- We introduce and evaluate trigger-word-based steganographic encoding, a novel scheme where signal placement depends on lexical context rather than fixed word positions.
- We demonstrate that LLM-based blind detection is non-functional at all dilution levels, with models applying fixed biases (always "yes" or always "no") regardless of whether hidden messages are present.

The remainder of this paper is organized as follows. Section 2 surveys related work on LLM steganography and AI safety. Section 3 describes our experimental methodology. Section 4 presents results across all three experiments. Section 5 discusses implications for AI safety and the nature of LLM steganographic capabilities, and section 6 concludes.

2 Related Work

LLM steganographic encoding. The most directly relevant line of work evaluates how well LLMs can produce steganographic text when prompted. Karpov [2026] introduced NEST, a systematic evaluation of acrostic steganography across 28 models and 10 difficulty levels. They showed that Claude Opus 4.5 achieves 92% per-number accuracy at sequence length $D{=}4$ but drops to 12% at $D{=}50$, establishing that acrostic accuracy degrades with message length. Zolkowski et al. [2025] extended this by comparing two encoding schemes—`sentence_letter` (acrostic) and `third_word_letter`—finding that models perform substantially worse on the latter. They also showed that models default to acrostic encoding when asked to steganographically encode without specifying a scheme. Karpov et al. [2025] explored reinforcement learning for steganography, showing that RL fine-tuning improves covert communication by roughly 20% for 3 bits of entropy, and that performance declines 33.5% on average as payload length increases. Unlike these works, we systematically vary dilution level across six schemes and measure encoding, extraction, and detection in a unified framework.

Steganographic collusion and AI safety. A parallel line of research examines steganography as a safety concern in multi-agent systems. Motwani et al. [2024] formalized secret collusion among AI agents, testing GPT-4 for steganographic message passing and finding up to 26% success rate in prompted settings. Mathew et al. [2024] demonstrated that steganographic collusion can emerge from misspecified reward incentives during training, not just from prompting, and found that standard mitigations such as paraphrasing are insufficient. Roger and Greenblatt [2023] showed that language models can be fine-tuned to hide reasoning in chain-of-thought outputs and proposed paraphrasing as a defense, which limits encoding to roughly 3 bits per kilobyte. These works establish

Table 1: Encoding schemes ordered by increasing signal dilution. Signal density indicates the approximate number of words per encoded character. The TRIGGER-WORD scheme density varies with the frequency of "the" in the generated text.

Scheme	Signal Density	Description
ACROSTIC	∼1 char/sentence	First letter of each sentence spells the message
EVERY-3TH-WORD	1 char/3 words	First letter of every 3rd word encodes a character
EVERY-5TH-WORD	1 char/5 words	First letter of every 5th word encodes a character
TRIGGER-WORD	∼1 char/8 words	Word after each "the" starts with the target letter
EVERY-10TH-WORD	1 char/10 words	First letter of every 10th word encodes a character
EVERY-20TH-WORD	1 char/20 words	First letter of every 20th word encodes a character

the safety motivation for understanding steganographic capabilities but do not characterize how those capabilities vary with signal dilution—the central question of our study.

Classical text steganography. Before the LLM era, text steganography relied on algorithmic methods such as synonym substitution, syntactic transformation, and format-based encoding [Ziegler et al., 2019]. Ziegler et al. [2019] proposed neural linguistic steganography using arithmetic coding with language models, achieving high capacity with low perplexity overhead. These methods differ fundamentally from prompted LLM steganography: they use the model as a tool for sampling steganographic text rather than relying on the model to understand and implement an encoding scheme. Our work focuses on the latter setting, which is more relevant to AI safety concerns about models spontaneously developing or following steganographic instructions.

3 Methodology

We design three experiments that jointly characterize how signal dilution affects LLM steganographic capabilities: encoding (can the model produce it?), informed extraction (can the model decode it?), and blind detection (can the model detect it?). Each experiment tests three frontier models across six encoding schemes of increasing dilution.

3.1 Encoding Schemes

We define six encoding schemes ordered by increasing signal dilution, measured as the number of words between consecutive signal-carrying positions. Table 1 summarizes the schemes.

The ACROSTIC scheme serves as a concentrated baseline: it aligns signal with sentence boundaries, a natural linguistic unit that LLMs encounter frequently in training data. The positional schemes (EVERY-3TH-WORD through EVERY-20TH-WORD) require the model to maintain a running word counter during generation or extraction—a capability known to be challenging for autoregressive transformers [Vaswani et al., 2017]. The TRIGGER-WORD scheme introduces a qualitatively different challenge: signal positions are determined by lexical context (occurrences of "the") rather than fixed positions, requiring the model to track both word identity and counting simultaneously.

3.2 Models

We evaluate three frontier models: GPT-4.1 (OpenAI API), CLAUDE SONNET 4.5 (via OpenRouter), and GEMINI 2.5 PRO (via OpenRouter). We use temperature 0.3 for encoding tasks (to allow creative text generation) and temperature 0.0 for extraction and detection tasks (to maximize determinism). Maximum token length is set to 1024 for encoding and 100 for extraction and detection responses.

3.3 Secret Messages

We use five secret messages of 5–6 characters: "HELLO", "FROST", "MAGIC", "DANCE", and "SILVER". These use common English initial letters to avoid biasing results toward rare-letter difficulty.

3.4 Experiments

Experiment 1: Encoding capability. We prompt each model to write natural-sounding text that encodes a secret message using a specified scheme. We provide explicit instructions describing the encoding rule (e.g., "write a paragraph where the first letter of every 5th word spells HELLO"). We then programmatically extract the hidden message from the generated text and measure accuracy. This yields 90 trials (3 models $\times$ 6 schemes $\times$ 5 messages).

Experiment 2: Informed extraction. We generate ground-truth steganographic texts programmatically, ensuring correct encoding is verifiable. We provide each model with the text and explicit extraction instructions (e.g., "extract the message encoded in the first letter of every 5th word") and measure extraction accuracy. This yields 90 trials.

Experiment 3: Blind detection. We present texts to each model without revealing the encoding scheme. For each model–scheme combination, we include 3 steganographic texts and 2 control texts (text without hidden messages), and ask the model to determine whether a hidden message is present. This yields 90 trials (3 models $\times$ 6 schemes $\times$ 5 texts).

3.5 Evaluation Metrics

Per-character accuracy. The fraction of character positions where the extracted character matches the target. For a target message of length n and extracted sequence $\hat{m}$, accuracy is $\frac{1}{n}\sum_{i=1}^{n}\mathbf{1}[\hat{m}_i = m_i]$. Random chance is $1/26 \approx 3.8\%$.

Exact match rate. The fraction of trials where the entire extracted message matches the target exactly.

True positive rate (TPR) and false positive rate (FPR). For blind detection, the fraction of steganographic texts correctly identified (TPR) and control texts incorrectly flagged (FPR).

3.6 Statistical Analysis

We use Mann-Whitney U tests [Mann and Whitney, 1947] to compare accuracy distributions between scheme pairs, as our sample sizes are small and we cannot assume normality. We compute Spearman rank correlations between dilution rank (ordinal: 1 = ACROSTIC through 6 = EVERY-20TH-WORD) and per-character accuracy to test for monotonic dilution–accuracy relationships. We report significance at $p < 0.05$ and $p < 0.01$ thresholds.

4 Results

4.1 Experiment 1: Encoding Capability

Table 2 presents per-character encoding accuracy across all model–scheme combinations. The dominant pattern is an abrupt cliff between ACROSTIC and all other schemes.

GPT-4.1 and CLAUDE SONNET 4.5 achieve perfect acrostic encoding (100% per-character accuracy, 100% exact match). GEMINI 2.5 PRO reaches 92% with some variance. Beyond acrostics, all models collapse: the highest non-acrostic accuracy is 43% (GEMINI 2.5 PRO on EVERY-3TH-WORD), but with a standard deviation of 0.52, indicating that the model occasionally produces correct output but cannot do so reliably. CLAUDE SONNET 4.5 is most affected, achieving 0% on EVERY-5TH-WORD and near-zero on most other diluted schemes.

Exact match rates are 0% for all models on all diluted schemes, meaning no model ever produced a fully correct diluted encoding in any trial.

Table 2: Per-character encoding accuracy (mean ± std) across models and schemes. All models achieve near-perfect ACROSTIC performance but collapse on diluted schemes. Best results per scheme in **bold**. Random chance is 3.8%.

Scheme	GPT-4.1	CLAUDE SONNET 4.5	GEMINI 2.5 PRO
ACROSTIC	**1.00** ± 0.00	**1.00** ± 0.00	0.92 ± 0.18
EVERY-3TH-WORD	0.23 ± 0.09	0.04 ± 0.09	**0.43** ± 0.52
EVERY-5TH-WORD	**0.31** ± 0.18	0.00 ± 0.00	0.15 ± 0.17
TRIGGER-WORD	**0.31** ± 0.18	0.14 ± 0.22	0.30 ± 0.24
EVERY-10TH-WORD	0.23 ± 0.09	0.07 ± 0.10	**0.31** ± 0.27
EVERY-20TH-WORD	0.23 ± 0.09	0.04 ± 0.09	0.23 ± 0.09

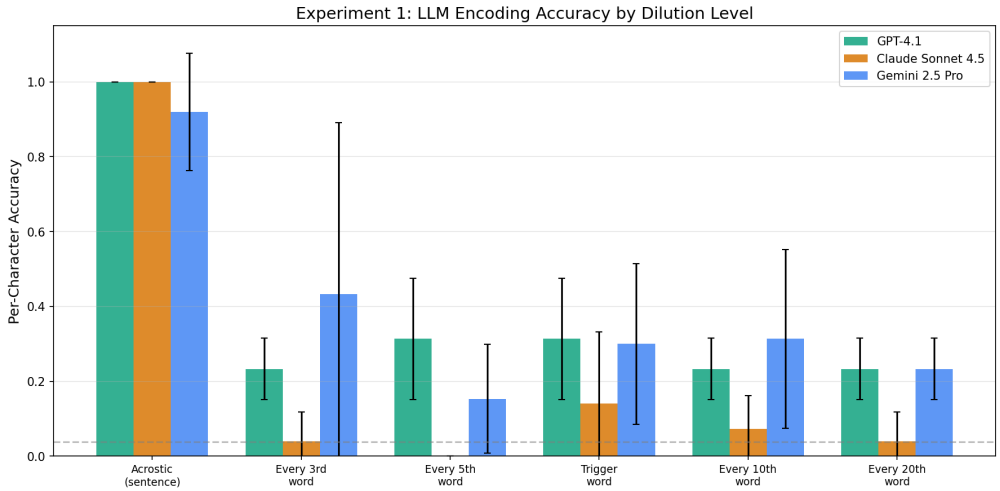

Figure 1: Encoding accuracy across dilution levels. All three models show a cliff effect: near-perfect ACROSTIC performance collapses to below 43% for every diluted scheme. CLAUDE SONNET 4.5 is most affected, falling to near-zero on all non-acrostic schemes. GEMINI 2.5 PRO shows high variance on EVERY-3TH-WORD (0.43 ± 0.52), indicating inconsistent performance.

Mann-Whitney U tests confirm that the acrostic–diluted gap is statistically significant for all models ($p < 0.01$ for GPT-4.1 and CLAUDE SONNET 4.5; $p < 0.01$ for most GEMINI 2.5 PRO comparisons). Spearman correlations between dilution rank and accuracy are $\rho = -0.47$ ($p = 0.008$) for GPT-4.1, $\rho = -0.43$ ($p = 0.017$) for CLAUDE SONNET 4.5, and $\rho = -0.32$ ($p = 0.088$) for GEMINI 2.5 PRO.

4.2 Experiment 2: Informed Extraction

Table 3 shows extraction accuracy when models are given ground-truth steganographic text and told the encoding rule.

Several patterns emerge. First, GPT-4.1 is the strongest extractor across all schemes, maintaining 35–55% accuracy on diluted schemes. This is well above the 3.8% random baseline but far from practical reliability. Second, extraction is easier than encoding for GPT-4.1: on EVERY-5TH-WORD, extraction accuracy (55%) exceeds encoding accuracy (31%) by 24 percentage points. Third, CLAUDE SONNET 4.5 degrades most sharply, dropping from 80% on ACROSTIC to 3% on EVERY-20TH-WORD. We observed that Claude frequently responded with meta-commentary ("I'll identify every 10th word...") instead of actually performing the extraction, suggesting difficulty following positional counting instructions. Fourth, TRIGGER-WORD extraction is difficult for all models, with GEMINI 2.5 PRO achieving only 15%.

Table 3: Per-character extraction accuracy (mean ± std) on programmatically generated ground-truth steganographic texts. GPT-4.1 retains moderate partial accuracy on diluted schemes. Best results per scheme in **bold**.

Scheme	GPT-4.1	CLAUDE SONNET 4.5	GEMINI 2.5 PRO
ACROSTIC	**1.00** ± 0.00	0.80 ± 0.45	0.84 ± 0.36
EVERY-3TH-WORD	**0.43** ± 0.33	0.19 ± 0.18	0.31 ± 0.22
EVERY-5TH-WORD	**0.55** ± 0.42	0.28 ± 0.30	0.27 ± 0.12
TRIGGER-WORD	**0.43** ± 0.17	0.27 ± 0.34	0.15 ± 0.17
EVERY-10TH-WORD	**0.39** ± 0.35	0.07 ± 0.10	0.23 ± 0.06
EVERY-20TH-WORD	**0.35** ± 0.36	0.03 ± 0.07	0.19 ± 0.12

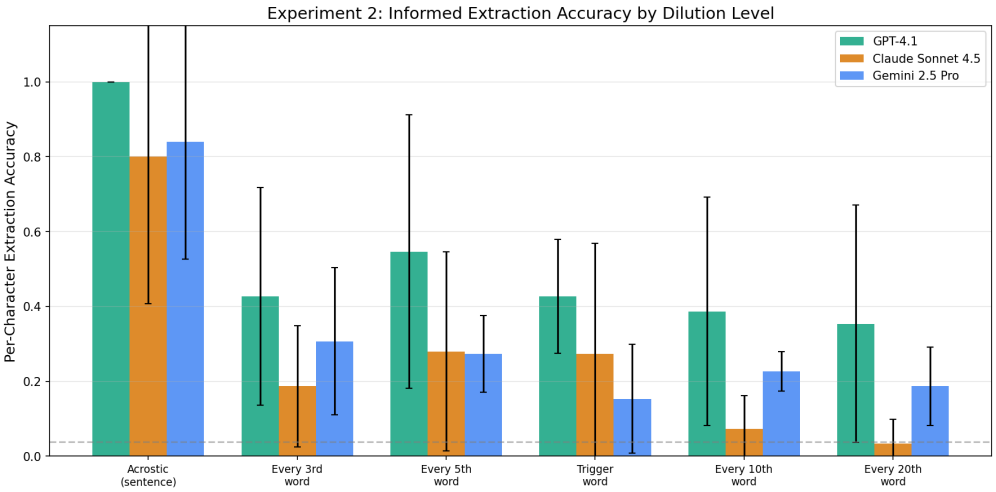

Figure 2: Informed extraction accuracy across dilution levels. GPT-4.1 maintains 35–55% partial accuracy on diluted schemes, well above random chance (3.8%) but far from reliable. CLAUDE SONNET 4.5 degrades most sharply, dropping from 80% on ACROSTIC to 3% on EVERY-20TH-WORD.

Spearman correlations between dilution rank and extraction accuracy are significant for all three models ($p < 0.01$), confirming a monotonic decrease.

4.3 Experiment 3: Blind Detection

Table 4 presents detection results. No model achieves any discriminability between steganographic and control texts.

GPT-4.1 and GEMINI 2.5 PRO always answer "yes" to the detection question, even for control texts containing no hidden message. GPT-4.1 frequently hallucinates patterns in controls, attempting first-letter extraction on random text and reporting spurious messages. CLAUDE SONNET 4.5 always answers "no," but—paradoxically—when it analyzes acrostic texts, it correctly identifies the first letters of each sentence and extracts the exact hidden message (100% extraction accuracy) while still concluding "DETECTION: NO." This reveals a conservative classification bias independent of the model's actual analytical ability.

Dilution level has no effect on detection because detection does not function at any level. The failure is not one of sensitivity to diluted signals but of calibration in the classification judgment itself.

Table 4: Blind detection performance. Each model applies a fixed response bias regardless of whether the text contains a hidden message, yielding zero discriminability. TPR = true positive rate, FPR = false positive rate.

Model	TPR	FPR	Discriminability
GPT-4.1	100%	100%	None (always YES)
CLAUDE SONNET 4.5	0%	0%	None (always NO)
GEMINI 2.5 PRO	100%	100%	None (always YES)

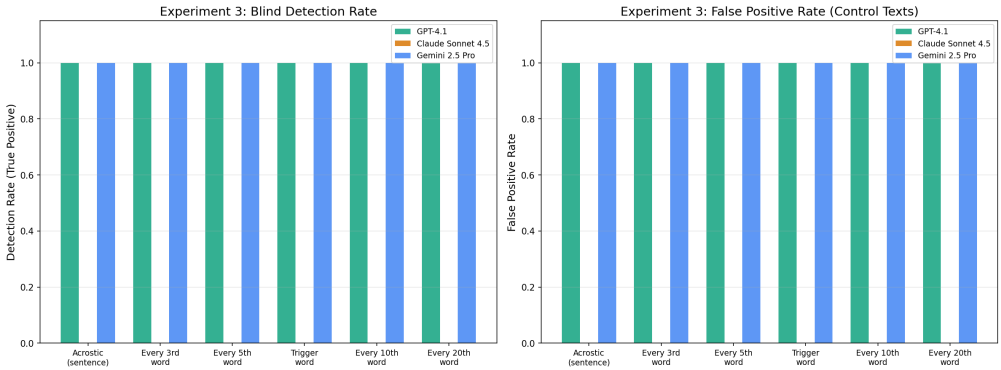

Figure 3: Blind detection rates by model and scheme. GPT-4.1 and GEMINI 2.5 PRO flag every text (including controls) as steganographic; CLAUDE SONNET 4.5 flags none. This pattern holds across all six encoding schemes, indicating fixed response biases rather than genuine detection capability.

4.4 Combined Dilution Curve

Figure 4 shows the overall dilution–accuracy relationship across all three experiments, averaged over models.

The cliff between ACROSTIC and all other schemes is the dominant pattern. Within the diluted schemes, there is a slight downward trend as dilution increases, but the differences between diluted schemes are small compared to the acrostic–diluted gap.

5 Discussion

5.1 The Cliff Effect: Pattern Memorization, Not Compositional Skill

The most striking finding is the abrupt collapse from near-perfect acrostic performance to near-random performance on any diluted scheme. We propose three complementary explanations.

First, **acrostics are a memorized pattern**. LLM training data contains extensive examples of acrostic poems, word puzzles, and writing exercises that use first-letter-of-sentence encoding. Models have learned "first letter of each sentence" as a specific, well-practiced skill. This is analogous to how models can perform arithmetic on numbers within their training distribution but fail on out-of-distribution formats [Vaswani et al., 2017].

Second, **counting is hard for autoregressive models**. Schemes like EVERY-5TH-WORD or EVERY-10TH-WORD require maintaining a running word counter during generation. Each token prediction must account for the current position in the counting sequence, a form of state tracking that transformers are known to struggle with when the state is not explicitly represented in the context.

Third, **sentences are natural boundaries**. Acrostics align signal with sentence breaks, which are already salient structural units for language models. Word-position schemes cut across sentence boundaries, requiring a fundamentally different type of attention to positional structure within running text.

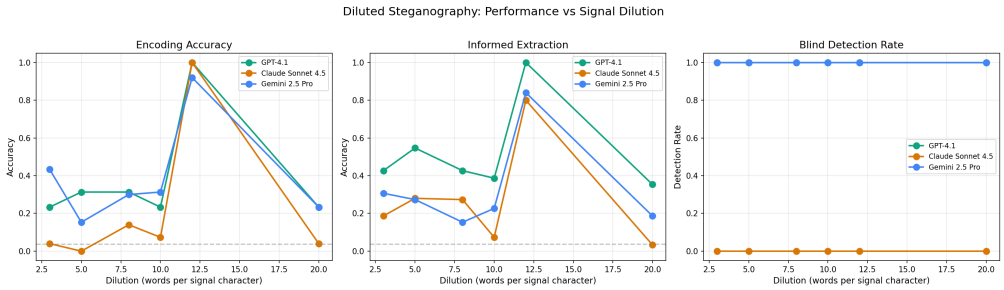

Figure 4: Combined dilution–accuracy curve across encoding and extraction experiments, showing the cliff between ACROSTIC and all diluted schemes. The gap between encoding and extraction accuracy is visible for diluted schemes, with extraction consistently higher, particularly for GPT-4.1.

The trigger-word scheme provides additional evidence for this interpretation. Although it avoids fixed positional counting, it requires tracking occurrences of a specific word ("the") and coordinating the next word's first letter—a task that combines lexical monitoring with constrained generation. Models find this comparably difficult to positional schemes.

5.2 Extraction vs. Encoding Asymmetry

GPT-4.1 shows a consistent asymmetry: extraction accuracy exceeds encoding accuracy on diluted schemes (e.g., 55% vs. 31% on EVERY-5TH-WORD). This makes intuitive sense. Encoding requires the model to *simultaneously* produce fluent text and satisfy positional constraints—a constrained generation problem. Extraction only requires reading existing text and identifying characters at specified positions—a comprehension problem. The asymmetry suggests that the bottleneck for diluted steganography is not understanding the rule but implementing it during generation.

CLAUDE SONNET 4.5 and GEMINI 2.5 PRO show a less clear asymmetry, partly because their extraction performance is also poor. CLAUDE SONNET 4.5 in particular often produced meta-commentary about its extraction strategy rather than actually extracting, suggesting a failure mode specific to instruction following under positional counting demands.

5.3 The Detection Failure

The complete failure of blind detection deserves separate discussion. The result is not that models are *bad* at detection—it is that they do not perform detection at all. Each model applies a fixed bias (always YES or always NO) regardless of input.

This is especially revealing in CLAUDE SONNET 4.5's case: when analyzing acrostic texts, it correctly extracts the hidden message while still reporting "no hidden message detected." The model can perform the analysis required for detection but cannot convert that analysis into a calibrated classification judgment. This suggests that the detection failure is not about analytical capability but about the model's uncertainty calibration and response formatting when asked a binary question about an unfamiliar task.

For AI safety, this means that LLM-based steganography monitors—approaches that ask a language model "does this text contain a hidden message?"—are fundamentally unreliable in their current form. Even if the monitor model can extract a hidden message when told where to look, it cannot make the prior judgment of whether to look.

5.4 Implications for AI Safety

Our results yield a mixed message for AI safety monitoring.

The good news. Current frontier models cannot produce diluted steganography. Even if a model were instructed to encode a message using every 10th word, it would fail. This limits the immediate risk of models spontaneously developing covert communication channels that evade simple pattern-matching detectors focused on acrostics.

The bad news. LLM-based monitors cannot detect steganography at any dilution level. If models eventually learn diluted encoding (through fine-tuning, tool augmentation, or architectural advances), current detection approaches would be blind to it. The defense and the attack are both broken, but the defense may be harder to fix because it requires solving a harder problem (detection without knowing the scheme) than the attack (encoding with a known scheme).

Recommendation. AI safety systems should not rely solely on LLM-based steganography detection. Statistical and algorithmic methods—entropy analysis, frequency analysis, information-theoretic bounds—may provide more robust detection than asking a language model "does this look suspicious?" Defense-in-depth approaches that combine multiple detection modalities are likely necessary.

5.5 Limitations

Sample size. We use 5 messages per condition, yielding 90 trials per experiment. Larger studies with 50–100+ messages per condition would provide tighter confidence intervals and more reliable effect size estimates. Our statistical tests are powered to detect the large acrostic–diluted gap but may miss smaller differences between diluted schemes.

Prompt sensitivity. We use a single, direct prompt design for each experiment. Chain-of-thought prompting, few-shot examples, or explicit counting scaffolds might improve performance on diluted schemes. Our results characterize default prompted behavior, not the ceiling of what models can achieve with optimized prompting.

Programmatic ground-truth texts. Our generated texts for Experiments 2 and 3 use formulaic structure and limited vocabulary, which may differ from natural steganographic text. Models might perform differently on more naturalistic cover texts.

Binary detection framing. Asking models for a YES/NO detection judgment may elicit response biases. A forced-choice design ("which of two texts contains a hidden message?") might yield more informative results by controlling for base-rate biases.

No fine-tuned models. All tests use off-the-shelf prompted models. Fine-tuned models [Roger and Greenblatt, 2023, Karpov et al., 2025] might exhibit different capability profiles, particularly for diluted schemes where the counting behavior could potentially be trained.

6 Conclusion

We presented a systematic study of how signal dilution affects LLM steganographic capabilities, testing three frontier models across six encoding schemes in a unified framework covering encoding, informed extraction, and blind detection. Our central finding is a cliff effect: all three models achieve 92–100% per-character accuracy on acrostic encoding but collapse to 0–43% on every diluted scheme, with 0% exact match across all diluted conditions. This transition is abrupt rather than gradual, suggesting that acrostic steganography is a memorized pattern rather than an instance of a general compositional steganographic capability. Blind detection fails entirely at all dilution levels, with each model applying a fixed response bias regardless of whether hidden messages are present.

These results carry a clear practical message: current LLMs can neither produce nor detect steganographic content beyond the well-practiced acrostic pattern. For AI safety, this means the immediate risk of covert LLM communication through diluted encoding is low, but so is the reliability of LLM-based steganography monitors.

Several directions remain open. Chain-of-thought prompting or explicit counting scaffolds may close part of the capability gap on diluted schemes. Fine-tuning on diluted encoding tasks would establish whether the cliff reflects a fundamental architectural limitation or a training data gap. Tool-augmented LLMs with access to word counters could bypass the counting bottleneck entirely. Finally, replacing binary detection with forced-choice or information-theoretic approaches may yield more calibrated detection systems. Understanding where the cliff can and cannot be moved is essential for anticipating the trajectory of steganographic capabilities as models continue to advance.

References

Artem Karpov. NEST: Nascent encoded steganographic thoughts. *arXiv preprint arXiv:2602.14095*, 2026.

Artem Karpov, Abraham Adeleke, Alexander Cho, and Juan Perez-Campanero. The steganographic potentials of language models. In *Building Trust Workshop at ICLR*, 2025.

Henry B Mann and Donald R Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.

Yohan Mathew, Piotr Matthews, Thomas McCarthy, Arsen Velja, Christian Schroeder de Witt, Joel Cope, and Nandi Sheridan Sheridan, Schoots. Hidden in plain text: Emergence and mitigation of steganographic collusion in LLMs. *arXiv preprint arXiv:2410.03768*, 2024.

Sumeet Motwani, Mikhail Baranchuk, Martin Strohmeier, Vrishab Bolina, Philip Torr, Lewis Hammond, and Christian Schroeder de Witt. Secret collusion among AI agents: Multi-agent deception via steganography. *arXiv preprint arXiv:2402.07871*, 2024.

Fabien Roger and Ryan Greenblatt. Preventing language models from hiding their reasoning. *arXiv preprint arXiv:2310.01405*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Zachary Ziegler, Yuntian Deng, and Alexander Rush. Neural linguistic steganography. *arXiv preprint arXiv:1909.01496*, 2019.

Artur Zolkowski, Sam Nishimura-Gasparian, Thomas McCarthy, Robert Zimmermann, and David Lindner. Early signs of steganographic capabilities in frontier LLMs. *arXiv preprint arXiv:2507.02737*, 2025.